

Homework 4 - Programming in Base R

Task 1: Conceptual Questions

Question 1: What is the purpose of the `lapply()` function? What is the equivalent `purrr` function?

The purpose of the `lapply()` function is to apply a function across different elements of a list. The regular `apply()` function only works with arrays, but `lapply()` allows us to work with lists that contain heterogeneous objects. The equivalent `purrr` function is `map()`.

Question 2: Suppose we have a list called `my_list`. Each element of the list is a numeric data frame (all columns are numeric). We want use `lapply()` to run the code `cor(numeric_matrix, method = "kendall")` on each element of the list. Write code to do this below! (I'm really trying to ask you how you specify `method = "kendall"` when calling `lapply()`)

The code to do this is `lapply(X = my_list, FUN = cor, method = "kendall")`

Question 3: What are two advantages of using `purrr` functions instead of the BaseR `apply` family?

Two advantages of using `purrr` functions instead of the BaseR `apply` family is that the `purrr` functions are more consistent across the board, whereas there are several differences in the BaseR `apply` family in terms of capitalization of terms (`fun` and `FUN`) and other minor differences. The `purrr` functions are also cleaner and easier to read than the `apply` family, and while they can be tough to master at first can allow for quicker and more readable code.

Question 4: What is a side-effect function?

A side effect function is a function that doesn't transform the data that we're working with, but instead creates something new from that data. An example of a side-effect function is `plot()` as we are taking the data and plotting it, but not changing anything with the data itself.

Question 5: Why can you name a variable `sd` in a function and not cause any issues with the `sd` function?

You can name a variable `sd` in a function and not cause any issues with the `sd` function because when we create functions we essentially also create a separate environment. Functions keep created objects separate, so naming something `sd` within a function doesn't affect `sd` outside the function, as the function object that was created is not kept after the function is run.

Task 2: Writing R Functions

Question 1: RMSE

When we start doing machine learning later in the course, a common metric used to evaluate predictions is called Root Mean Square Error (RMSE). Write a basic function (call it `getRMSE()`) that takes in a vector of responses and a vector of predictions and outputs the RMSE. If a value is missing for the vector of responses (i.e. an NA is present), allow for additional arguments to the `mean()` function (ellipses) that removes the NA values in the computation.

```
getRMSE <- function(responses, predictions, ...){  
  RMSE <- sqrt(mean(((responses - predictions)^2), ...))  
  RMSE  
}
```

Question 2: Testing RMSE function

Test your RMSE function using this data below. Repeat after replacing two of the response values with missing values (`NA_real_`). Test your RMSE function with and without specifying the behavior to deal with missing values.

```
#Testing data  
set.seed(10)  
n <- 100
```

```
x <- runif(n)
resp <- 3 + 10*x + rnorm(n)
pred <- predict(lm(resp ~ x), data.frame(x))

#Initial test
getRMSE(resp, pred)
```

```
[1] 0.9581677
```

```
#Replacing 2 response values with missing values
resp[1:2] <- NA_real_

#New test, with and without specification for NA
getRMSE(resp, pred)
```

```
[1] NA
```

```
getRMSE(resp, pred, na.rm = TRUE)
```

```
[1] 0.9661699
```

It works as intended!

Question 3: MAE

Another common metric for evaluating predictions is mean absolute deviation given by the MAE. Write a function called `getMAE()` that follows the specifications of the `getRMSE()` function.

```
getMAE <- function(responses, predictions, ...){
  MAE <- mean(abs(responses - predictions), ...)
  MAE
}
```

Question 4: Testing MAE function

Test your MAE function using this data below. Repeat after replacing two of the response values with missing values (NA_real_). Test your MAE function with and without specifying the behavior to deal with missing values.

```
#Testing data
set.seed(10)
n <- 100
x <- runif(n)
resp <- 3 + 10*x + rnorm(n)
pred <- predict(lm(resp ~ x), data.frame(x))

#Initial test
getMAE(resp, pred)
```

```
[1] 0.8155776
```

```
#Replacing 2 response values with missing values
resp[1:2] <- NA_real_

#New test, with and without specification for NA
getMAE(resp, pred)
```

```
[1] NA
```

```
getMAE(resp, pred, na.rm = TRUE)
```

```
[1] 0.8241201
```

It works as intended!

Question 5: Creating wrapper function

Let's create a wrapper function that can be used to get either or both metrics returned with a single function call. Do not rewrite your above two functions, call them inside the wrapper function (we would call the getRMSE() and getMAE() functions helper functions). When returning your values, give them appropriate names.

- The function should check that two numeric (atomic) vectors have been passed (consider `is.vector()`, `is.atomic()`, and `is.numeric()`). If not, a message should print and the function should exit.
- The function should return both metrics by default and include names. The behavior should be able to be changed using a character string of metrics to find.

```
vec_wrapper <- function(responses, predictions, metrics = c("RMSE", "MAE"), ...){
  if(!is.vector(responses) | !is.numeric(responses) | !is.atomic(responses)){
    stop("Error! Not a numeric (atomic) vector!")
  }
  if(!is.vector(predictions) | !is.numeric(predictions) | !is.atomic(predictions)){
    stop("Error! Not a numeric (atomic) vector!")
  }

  final <- list()
  if("RMSE" %in% metrics){
    final$RMSE <- getRMSE(responses, predictions, ...)
  }
  if("MAE" %in% metrics){
    final$MAE <- getMAE(responses, predictions, ...)
  }

  final
}
```

Question 6: Testing wrapper function

Test your new function using this data. Call it once asking for each metric individually and once specifying both metrics. Repeat with replacing two of the response values with missing values (NA_real_). Finally, test your function by passing it incorrect data (i.e. a data frame or something else instead of vectors)

```
#Testing data
set.seed(10)
n <- 100
x <- runif(n)
resp <- 3 + 10*x + rnorm(n)
pred <- predict(lm(resp ~ x), data.frame(x))

#Testing with normal data above
```

```
vec_wrapper(resp, pred, "RMSE")
```

```
$RMSE  
[1] 0.9581677
```

```
vec_wrapper(resp, pred, "MAE")
```

```
$MAE  
[1] 0.8155776
```

```
vec_wrapper(resp, pred, c("RMSE", "MAE"))
```

```
$RMSE  
[1] 0.9581677
```

```
$MAE  
[1] 0.8155776
```

```
#Replacing with NA  
resp[1:2] <- NA_real_  
#Retesting with NAs  
vec_wrapper(resp, pred, "RMSE")
```

```
$RMSE  
[1] NA
```

```
vec_wrapper(resp, pred, "MAE")
```

```
$MAE  
[1] NA
```

```
vec_wrapper(resp, pred, c("RMSE", "MAE"))
```

```
$RMSE  
[1] NA
```

```
$MAE  
[1] NA
```

```
#Now using na.rm  
vec_wrapper(resp, pred, "RMSE", na.rm = TRUE)
```

```
$RMSE  
[1] 0.9661699
```

```
vec_wrapper(resp, pred, "MAE", na.rm = TRUE)
```

```
$MAE  
[1] 0.8241201
```

```
vec_wrapper(resp, pred, c("RMSE", "MAE"), na.rm = TRUE)
```

```
$RMSE  
[1] 0.9661699
```

```
$MAE  
[1] 0.8241201
```

```
#Lastly, passing the function the wrong data type  
vec_wrapper(iris[,1:2], iris[,3:4])
```

Error in vec_wrapper(iris[, 1:2], iris[, 3:4]): Error! Not a numeric (atomic) vector!

Everything works as intended, and the last step gives me the proper error message!

Task 3: Querying an API and a Tidy-Style Function

Question 1: Using get() function

Use GET() from the httr package to return information about a topic that you are interested in that has been in the news lately (store the result as an R object). Note: We can only look 30 days into the past with a free account.

```
library(httr)
```

```
#Now building the URL and getting the info
my_url <- "https://newsapi.org/v2/everything?q=sustainability&from=2025-05-25&apiKey=f1558
url_info <- httr::GET(my_url)
```

Question 2: Parsing

Parse what is returned and find your way to the data frame that has the actual article information in it (check content). Note the first column should be a list column!

```
#Loading in packages
library(jsonlite)
library(tidyr)

#Parsing
parsed <- fromJSON(rawToChar(url_info$content))
my_info <- as_tibble(parsed$articles)
my_info
```

```
# A tibble: 99 x 8
  source$id $name author title description url urlToImage publishedAt content
  <chr>      <chr> <chr>  <chr> <chr>          <chr> <chr>          <chr>    <chr>
1 wired     Wired Alden~ Recy~ Unifi surv~ http~ https://m~ 2025-06-23~ "In th~
2 <NA>      Harv~ <NA>  Sust~ CEO Benede~ http~ https://h~ 2025-06-11~ "HANNA~
3 <NA>      Harv~ Kim G~ It's~ Strategies~ http~ https://h~ 2025-06-05~ "As co~
4 business~ Busi~ Joey ~ I sp~ I booked a~ http~ https://i~ 2025-06-18~ "Busin~
5 the-verge The ~ Emma ~ Shei~ The Europe~ http~ https://p~ 2025-05-27~ "An in~
6 <NA>      Hack~ Heidi~ Sust~ What if yo~ http~ https://h~ 2025-05-30~ "What ~
7 the-verge The ~ Justi~ AI c~ AI could s~ http~ https://p~ 2025-05-29~ "Resea~
8 <NA>      Harv~ Salon~ The ~ The value ~ http~ https://h~ 2025-06-05~ "Organ~
9 <NA>      Scie~ Jessi~ The ~ Trade impu~ http~ https://s~ 2025-06-17~ "In Ne~
10 <NA>      Cool~ Josh ~ Ruin~ Blanc Sing~ http~ https://1~ 2025-06-11~ "Blanc~
# i 89 more rows
```

Question 3: Writing a function

Now write a quick function that allows the user to easily query this API. The inputs to the function should be the title/subject to search for (string), a time period to search from (string - you'll search from that time until the present), and an API key.


```

my_key <- "f15582e470d142b3970b77b6f030ad8d"

api_query <- function(title, date, api_key){
  api_url <- paste0("https://newsapi.org/v2/everything?q=", title, "&from=", date,
                    "&apiKey=", api_key)
  api_info <- GET(api_url)
  api_parsed <- fromJSON(rawToChar(api_info$content))
  as_tibble(api_parsed$articles)
}

```

Question 4: Testing

Test your function for the title gamestop starting on May 19th, 2025.

```
api_query("gamestop", "2025-05-25", my_key)
```

```

# A tibble: 99 x 8
  source$id $name author title description url urlToImage publishedAt content
  <chr>      <chr> <chr> <chr> <chr>      <chr> <chr>      <chr>      <chr>
1 the-verge The ~ David~ A ni~ I'm standi~ http~ https://p~ 2025-06-05~ "Body ~
2 the-verge The ~ Brand~ The ~ Amazon's m~ http~ https://p~ 2025-06-20~ "Amazo~
3 business~ Busi~ fdemo~ Game~ GameStop a~ http~ https://i~ 2025-05-28~ "GameS~
4 <NA>      Gizm~ Kyle ~ Targ~ Check to m~ http~ https://g~ 2025-06-03~ "The S~
5 <NA>      Slas~ msmash Game~ GameStop i~ http~ https://a~ 2025-06-13~ "Cohen~
6 <NA>      Yaho~ Brad ~ Bitc~ The 2025 a~ http~ https://s~ 2025-05-28~ "Bitco~
7 <NA>      Gizm~ James~ Did ~ Maybe orde~ http~ https://g~ 2025-06-05~ "When ~
8 <NA>      Hipe~ Gabri~ Desa~ El gran dí~ http~ https://i~ 2025-06-05~ "El gr~
9 <NA>      Kota~ Ethan~ Stat~ Imagine yo~ http~ https://i~ 2025-06-05~ "Imagi~
10 <NA>      Gizm~ James~ Some~ There's on~ http~ https://g~ 2025-06-18~ "The S~
# i 89 more rows

```